

7th International Conference on Advances in Computing & Communications, ICACC-2017, 22-24 August 2017, Cochin, India

Dynamic load balancing algorithm for balancing the workload among virtual machine in cloud computing

Mohit Kumar^{a,*}, S.C.Sharma^b

^a*Research Scholar, IIT Roorkee, India*

^b*Professor, IIT Roorkee, India*

Abstract

Performance of the cloud infrastructure is highly depends upon the task scheduling and load balancing. Therefore number of load balancing algorithms and technique are proposed by researchers throughout the world whose aim is to distribute the workload fairly among all the virtual machine while attaining the objective. We developed a load balancing algorithm that minimize the makespan time and enhance the utilization ratio of cloud resources. Computational results (Fig. 3 to Fig. 5) shows that develop algorithm decrease the makespan time and enhance the utilization of resource compare to min-min algorithm, fcfs and shortest job first in all condition.

© 2017 The Authors. Published by Elsevier B.V.

Peer-review under responsibility of the scientific committee of the 7th International Conference on Advances in Computing & Communications.

Keywords Virtual machine; task scheduling; workload; makespan time; load balancing ;

1. Introduction

Cloud computing is emerging technology and next big step in the evolution of virtual computing in the field of IT within last few years. It provides the scalability, flexibility, on demand services and virtualization type of services over the internet to the user. As the number of user is increasing day-by-day in cloud environment, load balancing has been become a challenging problem for cloud service provider. To overcome this type of problem, many

* Corresponding author. Tel. +91-9759950380

E-mail address: mohit05cs33@gmail.com

algorithms have been proposed by researcher, but all the algorithms have their advantages and limitation. The main aim of load balancing is to utilize the cloud resource (combination of hardware and software) in such way so that throughput and resource utilization type of parameter should be maximum[1]. Load balancing helps to execute the task in minimum time and increasing the performance of the system i.e., user can get back response in shortest period of time [2]. Other challenges also exist in cloud environment like security [3], data loss, heterogeneity and high communication delay.

Load balancing is achieved in cloud environment in two steps: first one is to distribute the task among the node, second one is to monitor the virtual machine and perform the load balancing operation using task migration or virtual machine migration approach. The aim of task scheduling is to create a schedule and assigned each task to node (virtual machine) for specific time period so that all task are executed in minimum time span. Task scheduling is NP complete problem in the field of computer science because number of task and length of task change very rapidly in cloud environment. It is difficult to calculate all possible task-resource mapping in cloud environment and find an optimal mapping is not easy task. Therefore we need an efficient task scheduling algorithm that can distribute the task in effective way so that less number of virtual machine should be in overloaded or under loaded condition. After allocating the task to virtual machine, cloud task scheduler start to perform load balancing operation so that task can be transfer from overloaded virtual machine to under loaded virtual machine and all virtual machine should remain in balance condition. Three basic components are required for task scheduling in cloud environment as shown in Fig. 1.

User level phase First phase is called cloud user phase where users submit their jobs $J_1, J_2, \dots, J_n$ through the graphical user interface or web interface with service requirement in terms of quality of service (QoS), hardware, software etc.

Cloud task scheduler phase All the task scheduling and load balancing operation are performed in this phase. Job request handler forward the authentic request to task scheduler for further processing where matchmaker match all the tasks to corresponding virtual machine and scheduler assigned task/job to virtual machine. Task scheduler contains the information about all the virtual machine (idle or busy).

Cloud level phase Last phase of basic architecture of task scheduling in cloud environment is called cloud level phase. A datacenter contain the many host and each host contain the heterogeneous virtual machine as shown in Fig.1, number of virtual machine can increase and decrease at run time, its depend upon the capacity of host and number of upcoming user request. Cloud monitoring and discovering service (CMDS) is used to monitor the virtual machine (idle or busy state) and discover the resource information. The rest of the paper is organized as follows: Section 2 describes the related work in which we will discuss existing load balancing and task scheduling technique in cloud environment that is related to my research work, Section 3 describe the problem formulation, Section 4 describe proposed and develop load balancing algorithm, Section 5 describe the simulation tool and experimental results and last Section 6 conclusion and future work.

2 Related Work

Several static [4,5,6,7,8,9,10,12] and dynamic algorithm [11,13,14,15,16] has been proposed in last decade. Static algorithm needs advanced information about the number of task and information about available resource. There is no need to continuously monitor the resource when static algorithm is working. These types of algorithm gives better results only when there is low variation in upcoming workload. User-Priority Guided load balancing improve Min-Min Scheduling Algorithm was proposed by Huankai Chen et al.[5] in which n different length tasks ($T_1, T_2, T_3, T_4, \dots, T_n$) want to assigned to m heterogeneous resource ($R_1, R_2, R_3, R_4, \dots, R_m$), choose the minimum size task T and assigned to that resource which can execute the task minimum time in this algorithm. Suresh and vijayakarthick. P proposed an improved backfill algorithm (IBA) using balanced spiral (BS) method to improve the processing time of task [6], IBA algorithm provides the guarantee of quality of service in cloud environment but this algorithm do not provide better processing time when task comes randomly to cloud, to remove the limitation and achieve better quality of service with high resource utilization an algorithm IBA with EASY has been proposed for scheduling the task in cloud environment [7]. When user sends the request for the service, it also adds some quality of services parameter like deadline, priority, cost etc. M.Kumar and S.C.Sharma proposed a new algorithm that find

out the makespan time of tasks considering the priority to task i.e., priority task will execute firstly, after that non priority task will be executed [8]. M.Kumar et.al, proposed an algorithm that not only consider the priority of tasks but also consider cost of job [9].

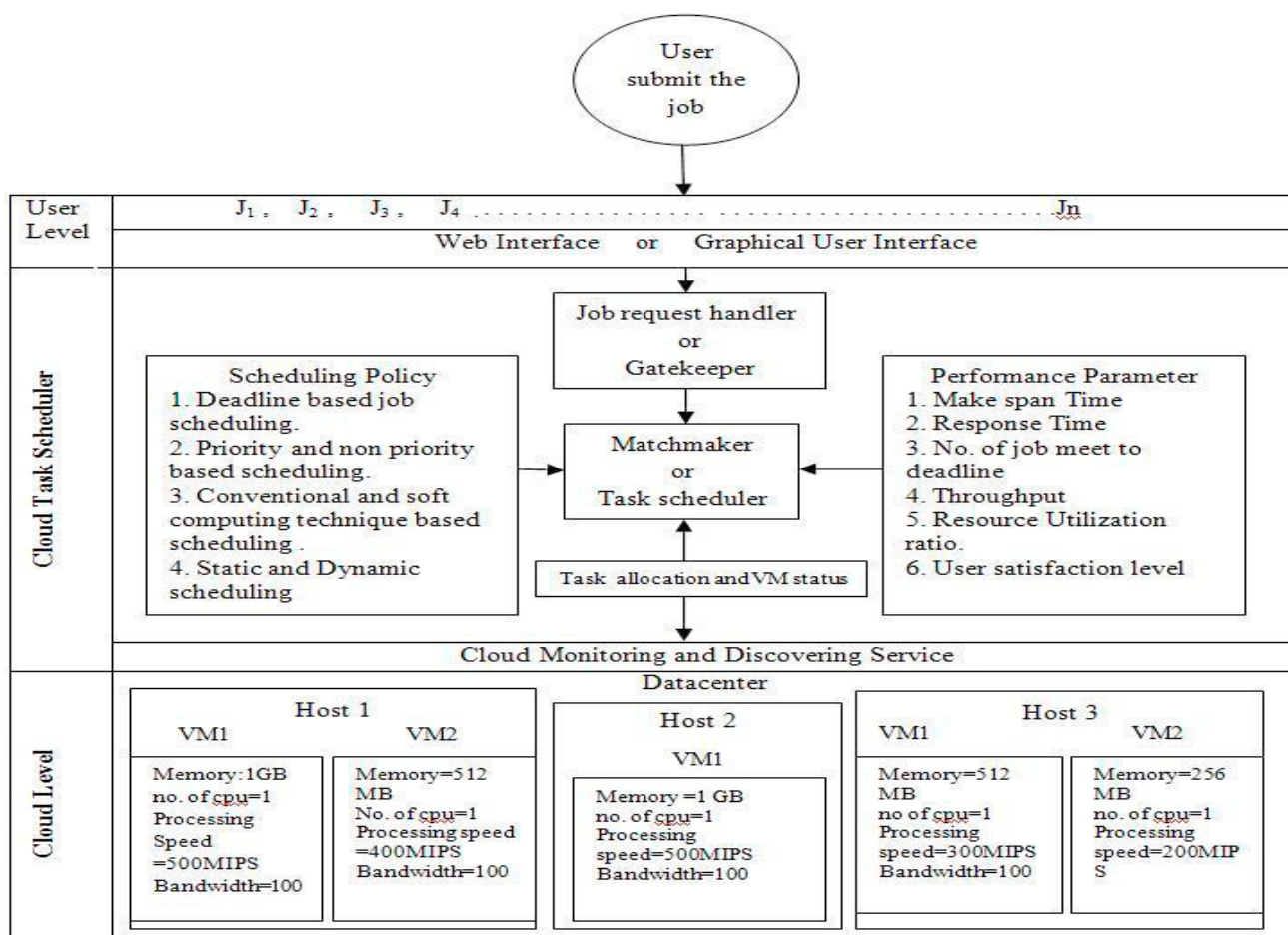


Fig. 1 Components required for Task scheduling in Cloud computing

J.Li et. al., [10] and B.Sahoo et. al., [11] both use the greedy approach to reduce the makespan time and execution time without using any load balancing approach (task migration or virtual machine migration), both the approach do not work well in real environment. Second [11] greedy based algorithm gives better results than [10] because this follows the dynamic approach. P.Samal and P.Mishra proposed round robin technique considering the parameter response time and resource utilization to solve the problem of load balancing in cloud environment [12] but algorithm don't minimize the response time and makespan time. All the discussed algorithm are not suitable for real time environment like cloud computing where load on node vary very frequently i.e., we cannot predict the upcoming load therefore we need a dynamic algorithm. There is no need of advanced information about the resource and task in dynamic algorithm because this type of algorithm continuously monitors the resource. A.Lakra and D.Yadav [13] proposed an algorithm to reduce turnaround time, cost and optimize throughput parameter also. A.thomas et.al.,[14] try to reduce makespan time using dynamic algorithm but unable to minimize makespan time completely for large number of task. X.Ren et. al.,[15] proposed a dynamic load balancing algorithm in which firstly predict the load on server using the load parameter cpu utilization, memory usage etc. after that start the load balancing operation. H.Chen et. al., [16] proposed an algorithm for utilization of resource and energy using the virtual machine migration approach. There are some other types of dynamic algorithms that use heuristic approach like max-min [17] algorithm and metaheuristic approach like ACO[18], PSO [19], ABC [20] etc to solve the task

scheduling problem in cloud environment but sometimes these type of algorithm trap in local optima. N.Tziritas et. al.,[21] proposed an algorithm for reducing the execution time and communication cost using the task migration technique. We have proposed and develop a dynamic load balancing algorithm considering the make span time and average resource utilization ratio as a parameter using the task migration approach.

3 Problem Formulation

Scheduling of n task into m node (virtual machine) is a NP Complete problem in the field of computer science. We have to schedule the entire task to node (virtual machine) in such a way that cloud user can execute their task in minimum makespan time and average resource utilization should be maximum. Cloud task scheduler receives N number of task request (job/cloudlet request) $T_1, T_2, T_3, T_4, \dots, T_N$. In this paper, we are not considering quality of service parameter like deadline, priority, cost etc. All the tasks are non priority basis and independent in nature, every task has task length TL_i . Which is expressed in MI (million instructions) each task requires p processing speed, q number of cpu, r amount of main memory and required bandwidth B in MBPS. Cloud task scheduler contains the information about the M heterogeneous virtual machine (different processing speed of processor, number of cpu, memory, bandwidth etc.). $VM_1, VM_2, VM_3, \dots, VM_M$. Firstly we calculate the capacity of individual virtual machine and capacity of all virtual machine using the formula

$$C_{VM} = p * q \quad (1)$$

p =processing speed of processor (cpu) in million instruction per second

q = number of cpu are busy to execute the task.

$$\text{Capacity of all virtual machine } C = \sum_{j=1}^M C_{VM_j} \quad (2)$$

Load Information on Virtual Machine: Cloud task scheduler allocate the task to virtual machine, every virtual machine has a queue to store the load. Total length of queue in virtual machine represents the load on that virtual machine. Load at a virtual machine can be calculated as

$$LVM_{i,t} = K * TL_{i(t)} / S(VM_{i,t}) \quad (3)$$

Where $K=1, 2, 3, \dots, N$ tasks

$S(VM_{i,t})$ is defined the service rate of virtual machine at time t , that can be expressed in the form of processing power p and number of cpu q as $p * x(t)$ where $x=1, 2, 3, \dots, q$.

Load on a virtual machine at a time t can be calculated as the number of task on particular virtual machine is divided by service rate of virtual machine.

$$\text{Total load at all virtual machine } L = \sum_{j=1}^M LVM_j \quad (4)$$

If upcoming workload at the datacenter (all virtual machine) is more than the capacity of datacenter then datacenter cannot handle the entire upcoming request so either discard the upcoming task request or increase the virtual machine using the elasticity concept. If upcoming workload is less than the datacenter capacity then we find the load at each individual virtual machine and find out the number of virtual machine are in overloaded or under loaded condition. If any virtual machine is over loaded then we transfer task of overloaded virtual machine to under loaded virtual machine so that all the tasks can execute their execution in minimum time.

Task transfer time can be calculated $TT = \text{length of task (TL)} / \text{Bandwidth (B)}$

We find execution time of task T_i on virtual machine VM_j

$$T_{exej} = \sum_i^N E_{ij} * TL_i / p * q \quad (5)$$

$E_{ij}=1$ if task T_i is assigned to virtual machine VM_j otherwise its value is 0.

Our objective is to find out the makespan time that contain task execution time and task transfer time (if any task is transfer from overloaded virtual machine to under loaded virtual machine)

$$MST = \max \{ \sum_{j=1}^M T_{exej} \} \quad (6)$$

4 Proposed load balancing algorithm

We have proposed a dynamic load balancing algorithm whose objective is to reduce the makespan time and increase the average resource utilization ratio in cloud environment as shown in Fig. 2. We generated N number of task and M number of virtual machine with different processing power in terms of processor speed in MIPS, RAM etc. Length of task is generated randomly (range of task between 20000MI to 400000 MI) and sort the task as well as

virtual machine in decrease order of their task length and processing speed and start to allocate the task to virtual machine in FCFS order. We create M number of array that contains the allocated task Id of each virtual machine. Once task has been assigned to virtual machine then we start load balancing operation at virtual machine and find number of task, length of each task assigned to individual virtual machine and calculate the load on individual virtual machine, total load on datacenter at a particular time t using the equation 3 and 4. We calculate capacity of virtual machine and datacenter using the equation 1 and 2 and check the condition that upcoming load on all virtual machine is less than to capacity of datacenter. If this condition is false then load balancing operation is not possible on exist virtual machine i.e., cloud task scheduler boot new virtual machine using the elasticity concept. If condition is true then load balancing is possible in existing cloud infrastructure.

We find the excepted processing time of each task at virtual machine after that find out number of over loaded, under loaded and balanced virtual machine. We assume that a virtual machine is under loaded condition if its utilization is less than 25% of its capacity and virtual machine will be overloaded condition if its utilization is more than 80 % of its capacity. We defined two variable UM and OM that represent 25% of capacity of virtual machine and 80% capacity of virtual machine and compare with processing capability of each virtual machine. Find out the total number of overloaded virtual machine, under loaded virtual machine, balanced virtual machine and information about the total number of task hold by individual virtual machine and sort the overloaded virtual machine to decreasing order of task ID and under loaded virtual machine to increasing order. After that check the condition that any virtual machine is under loaded condition, if yes then transfer the task from overloaded virtual machine to under loaded virtual machine until under loaded virtual machine reach to threshold limit of overload. After transferring the task check again load balancing condition and transfer the task if any virtual machine is not satisfied the overloading condition. We implemented task migration techniques, calculate the task migration time and added to makespan time of that particular virtual machine who is handling (executing) the task. Finally we submit the entire task and find out the makespan time using the proposed algorithm.

5 Simulation tool and experimental results

We have proposed and implement the dynamic load balancing algorithm for minimize the makespan time and increase the resource utilization ratio of task using the cloudsim platform. Experimenting new technique in real cloud environment is not possible practically because some experiment compromise the end user quality of service. Therefore we need a good simulator for experimental purpose. Cloudsim simulator is basically used for task scheduling and load balancing in cloud environment.

5.1 Makespan Time Calculations

We run the simulation more than two hours on different number of task (task range 10 to 50) with random length and find out the result using the space shared policy [22] in cloudsim. At the starting, consider 5 virtual machine of different processing power (200 to 1000 MIPS), range bandwidth of virtual machine was 100 to 1000 MBPS, number of cpu for each virtual machine was 1 as shown in Table 1 and consider 10 to 50 tasks with length 20000MI to 400000 MI as shown in Table 2. Results of Table 1 and Table 2 are shown in column 1 of Fig. 3, x axis represent the number of task and y axis represent the makespan time of task in second in Fig. 3. Task is allocated to all the virtual machine as per scheduling algorithm, after that start to monitor the virtual machine and find that virtual machine 5 (VM Id 4) is overloaded condition and no virtual machine is under loaded condition therefore we transfer the task of overloaded virtual machine to balance virtual machine, task will be transfer to virtual machine 1 and task migration time is also added. Finally all tasks will be executed 505 second. We increase number of task from 10 to 15 and randomly choose the task length between 20000 to 200000 MI. We reduce the task length because if we generate the 15 task of length 20K to 400K and allocate to 5 existing virtual machine then all the virtual machine will be overloaded condition and cloud task scheduler need to boot more virtual machine here we are working on load balancing problem so we reduce the length of task. Sometimes Min-Min algorithm makespan time is close to proposed algorithm (LBA) because Min-Min algorithm follow the heuristic approach and its resource utilization ratio is low compare to proposed LBA (no task is allocated to virtual machine 5). We increase the number of task up to 50 with different length and apply the same algorithm on task, results are shown in fig. 3.

5.2 Average Resource Utilization ratio (ARUR) The main purpose of load balancing is keeping the resource busy as much as possible. Average resource utilization ratio is calculated using the formula

$$ARUR = (\text{mean time/makespan}) * 100 \quad (7)$$

Where mean time = $\sum$ Time taken by resource (VM_j) to finish all the job/number of resource where $j = \{1, 2, 3, \dots, M\}$. The range of average resource utilization ratio is 0 to 1, maximum value for ARUR is 1 (resource utilization is 100%) and worst value is 0 (resource is in ideal condition).

Symbol Notation

UVM[] = under loaded virtual machine array, BVM[] = balanced virtual machine array, OVM[] = overloaded virtual machine array, UM = under loaded machine, OM = overloaded machine.

Task scheduling Operation

1. Generate N number of cloudlet (Task) and sort them in decreasing order.
2. Generate M number of virtual machine, Sort in decreasing order of their processing speed.
3. For $\forall T_i \in 0$ to $N-1$ and virtual machine $R_j \in 0$ to $M-1$.
4. Start to assign the task in First come first serve order.

End VM for loop, End task for loop

Load Balancing Operation

5. Start the for loop $i=0$ to $VmSize-1$ and cloudlet loop $k=0$ to $CloudletSize-1$
6. Find the load L and capacity of all virtual machine using the equations.
7. If $(L > C)$, load balancing operation is not possible, we have to use elasticity, otherwise start to monitor each VM.
8. Find out the number of under loaded, balanced and overloaded VM.

// UM, OM are variable $UM = .25 * C_{VM}$, $OM = .8 * C_{VM}$,

9. Find the VM on which we want to transfer the task,
Sort OM in decreasing order and UM in increasing order

Task Transfer operation

- 10 While $OM \neq \emptyset \ \&\& \ UM \neq \emptyset$

For loop for OM

If OM & UM is exist then transfer the task from OM to UM

$T_i \rightarrow VM_j$ | Until load at $VM_j \leq OM \parallel VM_j \geq UM$

Also calculate transfer time of task ($TL_i/\text{bandwidth}$).

11. After the task transfer check the status of each VM, if any virtual machine is still overloaded condition, repeat load balancing operation again.

Fig. 2 Proposed Load balancing algorithm

Table 1 VM properties

Table 2 Task properties

VM Id	VM MIPS	VM Image size	Memory	No. of CPU	VMM
0	1000	1000	512	1	Xen
1	800	1000	512	1	Xen
2	600	1000	256	1	Xen
3	400	1000	512	1	Xen
4	200	1000	256	1	Xen

Task id	Length	File size	Output file	No. of CPU
0	116228	300	300	1
1	52118	300	300	1
2	33734	300	300	1
3	67488	300	300	1
4	290543	300	300	1
5	60246	300	300	1
6	366852	300	300	1
7	73891	300	300	1
8	197914	300	300	1
9	54199	300	300	1

This paper focuses the parameter makespan time and ARUR. We will calculate average resource utilization ratio of proposed algorithm using the equation 7 and compare the results with existing algorithm. We consider the value of Table 1 and Table 2 for resource and number of task, after that we apply the Proposed load balancing algorithm to

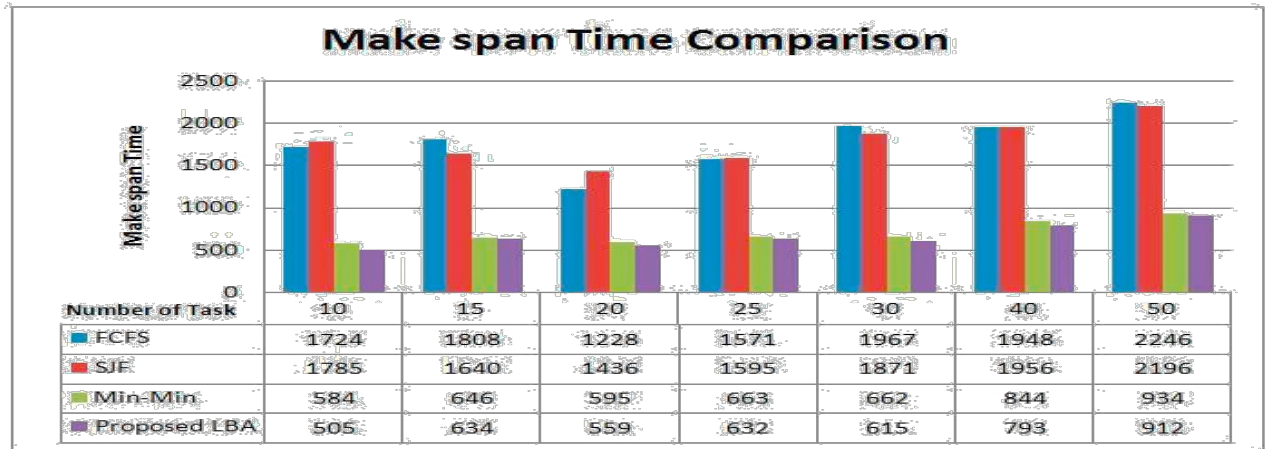


Fig. 3 Comparison of makespan time for proposed load balancing algorithm vs FCFS, SJF, Min-Min at 5 VM

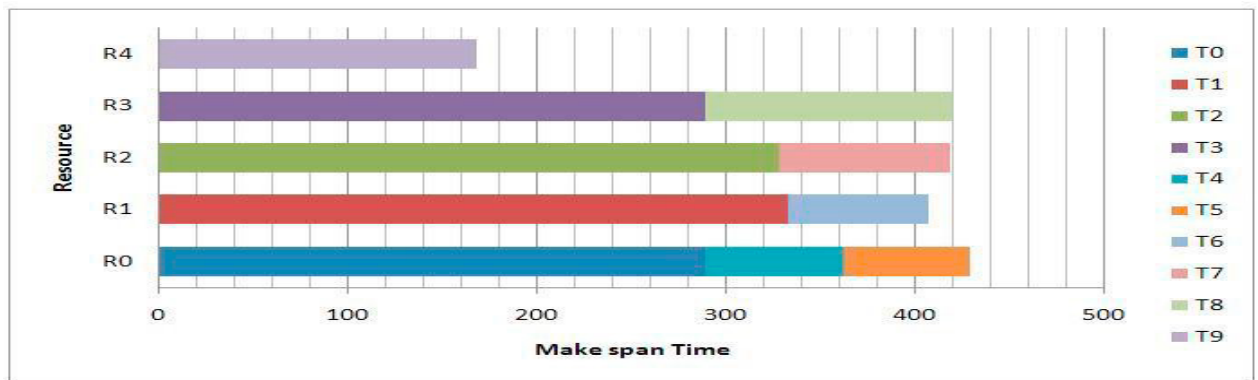


Fig. 4 Gantt chart of task allocation to 5 VM using the proposed load balancing algorithm

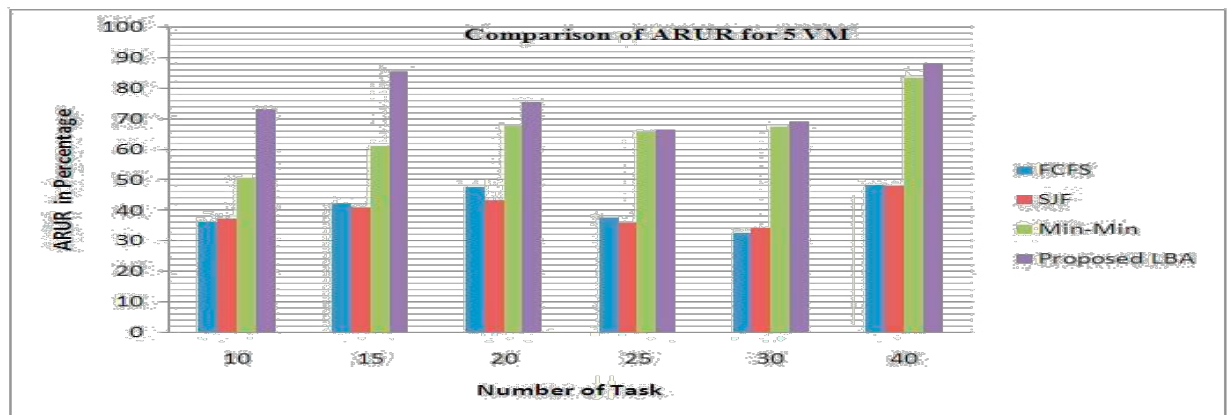


Fig. 5 Comparison of ARUR for proposed LBA vs FCFS, SJF, Min-Min

find the average resource utilization ratio. Gantt chart in Fig. 4 represent proposed load balancing algorithm allocated the tasks to virtual machine in such a way that no virtual machine in overloaded or under loaded condition. Calculated ARUR of proposed load balancing algorithm is 73 %. We apply the Min-Min algorithm on same data value and get ARUR 50.8 % because resource R3 and R4 in idle condition. When we apply FCFS and SJF algorithm on Table 1 and Table 2 and find that average resource utilization ratio of FCFS is 36.3 and SJF is 37.4.

Computational results in Fig. 5 shows that ARUR of proposed algorithm is approximately (average) 30 % more than FCFS, SJF algorithm and 10 % more than Min-min algorithm. x axis represent the number of task and y axis represent the ARUR in percentage of task in second in Fig. 5.

6. Conclusion and Future Work:

We developed a dynamic load balancing algorithm, whose objective is to utilize the cloud resource in effective way. There are lots of algorithm exist in cloud environment like heuristic based algorithm, metaheuristic based algorithm, conventional approach based algorithm etc. Each algorithm work on different parameter like makespan time, execution time, response time, resource utilization, throughput etc. and optimize the parameter based on objective function. We proposed a conventional based approach to balance the load in cloud environment using the task migration approach. This algorithm not only minimizes the makespan time but also decrease the possibility of overloaded and under loaded of a virtual machine. Experimental results shows that under all possible condition proposed algorithm reduce the makespan time and increase the average resource utilization ratio compare with FCFS, SJF and Min-Min shown in Fig. 3 to Fig. 5. We did not consider priority of task, deadline of task and other quality of service (QoS) parameter in this paper.

References

- [1]. Ren et al., "The load balancing algorithm in cloud computing environment," in International Conference on Computer Science and Network Technology, Changchun, China, 2012.
- [2]. J. Bhatia et al., "HTV Dynamic Load Balancing Algorithm for Virtual Machine Instances in Cloud," in International Symposium on Cloud and Services Computing, Mangalore, KA, 2012.
- [3]. M. Kumar and N. Roberts. (2013, August 2). "A technique to reduce the economic denial of sustainability (EDoS) attack in cloud"[Online]. Available searchdl.org/public/book_series/AETS/7/94.p.
- [4]. Mondal, R. Kumar, E. Nandi, and Debabrata Sarddar, "Load Balancing Scheduling with Shortest Load First," *International Journal of Grid and Distributed Computing* 2015.
- [5]. Chen et al., "User-priority guided min-min scheduling algorithm for load balancing in cloud computing," in National Conference on Parallel Computing Technologies., Bangalore.,KA, 2013.
- [6]. Suresh.A and Vijayakarthick.P, "Improving scheduling of backfill algorithms using balanced spiral method for cloud metascheduler," in International Conference on Recent Trends in Information Technology., Chennai., TN, 2011.
- [7]. K.Dubey et al., "A Priority Based Job Scheduling Algorithm Using IBA and EASY Algorithm for Cloud Metascheduler," in *International Conference on Advances in Computer Engineering and Applications*. Ghaziabad., India, 2015.
- [8]. M.Kumar and S.C.Sharma., "Priority Aware Longest Job First (PA-LJF) algorithm for utilization of the resource in cloud environment," *Computing for Sustainable Global Development (INDIACom), 2016 3rd International Conference on*. IEEE, 2016.
- [9]. M. Kumar, K.Dubey and S.C.Sharma, "Job Scheduling Algorithm in Cloud Environment Considering the Priority and Cost of Job," *Proceedings of Sixth International Conference on Soft Computing for Problem Solving*. Springer, India, 2017.
- [10]. J.Li et., "An greedy-based job scheduling algorithm in cloud computing," *Journal of Software*, 2014.
- [11]. B. Shao et al., "Analysing the Impact of Heterogeneity with Greedy Resource Allocation Algorithms for Dynamic Load Balancing in Heterogeneous Distributed Computing System," *Int J Comput Appl*, Jan. 2013.
- [12]. P. Samal and P. Mishra "Analysis of variants in Round Robin Algorithms for load balancing in Cloud Computing," *International Journal of Computer Science and Information Technologies*, 2013.
- [13]. A. Lakra, and D. Yadav, "Multi-Objective Tasks Scheduling Algorithm for Cloud Computing Throughput Optimization." *Procedia Computer Science*, 2015.
- [14]. A. Thomas et. al., "Credit Based Scheduling Algorithm in Cloud Computing Environment." *Procedia Computer Science*, 2015.
- [15]. X. Ren et al., "A Dynamic Load Balancing Strategy for Cloud Computing platform based on Exponential Smoothing Forecast," in International Conference on Cloud Computing and Intelligence Systems., Beijing., China, 2011.
- [16]. H. Chen et al., "Towards energy-efficient scheduling for real-time tasks under uncertain cloud computing environment," *J.Syst. Software*, Jan. 2015.
- [17]. K.Navdeep and K.Kaur, "Improved Max-Min Scheduling Algorithm.," *IOSR Journal of Computer Engineering*, vol. 17, May 2015.
- [18]. E.Pacini et al., "Balancing throughput and response time in online scientific Clouds via Ant Colony Optimization (SP2013/2013/00006)," *Advances in Engineering Software*, June 2015.
- [19]. F.Ramezani and F.Khadeer Hussain, "Task-based System Load Balancing in cloud computing using Particle Swarm Optimization" *Int J Parallel Prog*, Oct. 2013.
- [20]. D. Babu and P. Venkata, "Honey bee behavior inspired load balancing of tasks in cloud computing environments," *Appl Soft Comput*, May 2013.
- [21]. N. Tziritas., et al., "On minimizing the resource consumption of cloud applications using process migrations." *Journal of Parallel and Distributed Computing*, 2013.
- [22]. Himani and H. Sindhu, "Comparative analysis of scheduling algorithms of Cloudsim in cloud computing." *International Journal of Computer Applications*, 2014.